

Conflict as First-Class Object: Two-Level Conflict Handling in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 24 April 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the architectural commitment to conflict as first-class substrate object and the two-level handling pattern that makes the commitment operationally tractable, so that downstream work can adopt or argue against the commitment without ambiguity.

Abstract

The Coordination Knowledge Substrate (CKS) design pattern names "conflict-preserving" as one of its six architectural commitments. The commitment is more specific than its short name suggests: contradictions in substrate content are not merely retained as history, they are promoted to first-class substrate objects with their own identity, provenance, and addressability — and the commitment is operationally enforced through a two-level handling pattern. At the substrate level, contradictions persist by default and are never collapsed by automated processes. At the cell level, cells executing over substrate content with active contradictions resolve them according to human-authored orchestration rules, and the resolution decisions are themselves recorded as additional substrate content rather than as mutations of the underlying contradicting content. This note formalizes the commitment as the source paper uses it: it states the architectural promotion move, the two-level pattern and the coupling between its halves, the provenance requirements that distinguish first-class contradictions from mere accumulation of inconsistent content, the contrast with the detect-resolve-forget pattern, and the operational test for whether a given system instantiates the commitment.

1. The architectural promotion of contradiction

Most coordination systems treat contradictions as errors to be resolved before further processing. Contradictions detected in retrieved documents are filtered before the model sees them; contradictions surfaced in an LLM's output are reconciled by a judge model or an additional clarification turn; contradictions in a knowledge base are flagged for cleanup. The shared assumption is that the system's correct behavior in the presence of a contradiction is to remove it: the contradiction's existence is a defect of the system's state, not a property of the world the system is trying to coordinate over.

CKS commits to the opposite stance, named as architectural commitment #2 in §2.1 of the source paper and developed in §5: contradictions are *first-class addressable objects* in the substrate, with their own identity and provenance, persisted by default. A contradiction has substrate existence on the same terms as a decision, an entity, or a rule: it can be looked up, pointed at, written about, and re-encountered across sessions; it is part of the state the substrate carries, not a transient signal in the substrate's input stream.

The reason is not that contradictions are sometimes worth keeping for the record. The reason is that the reasoning trail captured in coordination work often *hinges* on contradictions deliberately not collapsed: two valid interpretations of an ambiguous requirement held in parallel; dissent from a prior decision recorded as a contradicting position even when the decision stands; regulated coordination work in which the dispute is itself part of the artifact of value, where an audit reconstructing what was decided needs to see what was contested, not only what won. Detect-resolve-forget patterns destroy exactly this content; the CKS commitment preserves it as substrate state.

2. The two-level handling pattern

If contradictions are never collapsed, how do cells that need to act on substrate content actually act? CKS answers with a two-level handling pattern, developed in §5.3 of the source paper. The pattern is the most important architectural move in the conflict-preservation commitment, because it is what keeps the commitment from collapsing into either of two unworkable alternatives — a system that can never resolve anything, or a system that resolves things the same way every other system does.

Level 1 — Substrate-level preservation. Contradictions in substrate content are recorded as substrate content. They have writers, timestamps, and (where applicable) rationale. They are addressable: another piece of substrate content can refer to them, ask about them, or describe their relationship to other content. They persist until a human exercising authority over the substrate, or a human-authored orchestration rule explicitly authorizing collapse for the specific case, resolves them. They are never collapsed by an LLM operation, an automated process, or a runtime middleware layer. §5.3 of the source paper states this absolutely: no LLM operation over the substrate may silently merge, discard, or force resolution of preserved contradictions, regardless of delegation level.

Level 2 — Cell-level resolution under orchestration rules. Cells executing over substrate content with active contradictions resolve them according to the orchestration rules governing the cell. The rules may direct the cell to preserve and skip, to surface the contradiction to a participant, to delegate it to another substrate, to defer with a recorded outcome, or to apply a pre-specified resolution logic under specified conditions. Whatever the cell does, two properties hold. First, the resolution is a *cell behavior*, not a substrate mutation: the underlying contradicting content remains in the substrate after the cell finishes. Second, the resolution decision is itself substrate content — addressable, traceable, with its own writer (the cell, executing under a named rule), its own timestamp, and its own reference to the rule that authorized it.

The coupling. The two levels are halves of a single commitment, each load-bearing for the other. Substrate-level preservation makes cell-level resolution decisions auditable: because the underlying

contradiction persists, any later participant can see what the cell resolved and what it resolved against, and can contest the decision by adding new substrate content rather than by reconstructing a destroyed history. Cell-level resolution under rules makes substrate-level preservation operationally tolerable: cells do not block on every contradiction, so the substrate can hold contradictions indefinitely without halting the work the cells exist to perform. Either level alone produces an unworkable architecture; both together produce a workable one.

The architectural constraint the commitment imposes, as §5.3 puts it, is not on *whether* contradictions get resolved — they often do — but on *who decides* the resolution logic and *where the resolution lives*. Humans decide, either directly by editing substrate content or indirectly by authoring the rules cells execute under. Resolutions live as new substrate content layered over the contradicting content, not as deletions of it.

3. Contrast with detect-resolve-forget

The dominant alternative pattern in current LLM agent frameworks and in adjacent contradiction-handling research has a handle worth naming, and §5.4 of the source paper supplies it: **detect-resolve-forget**. The pattern detects a contradiction at inference time, resolves it in-session through clarification, majority voting, or judge-model arbitration, and discards the alternatives once the session ends. The 2024–2025 literature instantiates the pattern in RAG contradiction-detection systems with context validators, in explanation-generation work that succeeds by flipping a contradiction label to entailment, in clarification-question agents that overwrite prior estimates with refined ones, and in stress-testing frameworks that synthesize contradictions to evaluate detection-and-resolution capability rather than to preserve substrate state. Across these lines, the commitment is the same: contradictions are transient, in-session, and reconciled before the system emits.

The detect-resolve-forget pattern is incompatible with the CKS conflict-preservation commitment for three reasons.

First, it violates conflict preservation as a first-class architectural commitment. Substrate-level preservation requires that the contradiction persist as substrate state after any resolution decision (§5.3). Detect-resolve-forget defines success as the contradiction's disappearance: the resolved version is what remains, the alternatives are what is forgotten. The two stances are opposite at the architectural layer, not adjustable variants of the same stance.

Second, it produces non-addressable writes. The path-retraceability vocabulary §3.1 imports from Rajabi and Kafaie (2022) requires that the substrate carry an explicit, traversable record of how each piece of content came to be. A detect-resolve-forget operation produces a single surviving version with no record of what was discarded, who discarded it, on what evidence, or against what alternative. The resolution decision exists only as the surviving version; it is unaddressable because there is no remaining structure to address. CKS-compliant resolution, by contrast, leaves both the contradiction and the resolution decision as addressable substrate content, each with its own provenance.

Third, it relocates resolution authority from human or rule-governed to automated. Detect-resolve-forget resolves contradictions inside the LLM's reasoning, inside a judge-model's

arbitration, or inside a context validator's filter. None of these loci are human-governed in the sense the source paper commits to: the resolution is made by an automated process, with no human authority over the specific case and no rule a human authored at design time that the process is executing. Substituting automated resolution for either human or rule-governed resolution violates the human-governed commitment in addition to conflict preservation.

The contrast is architectural, not efficacy-based, and §5.4 is careful on this point. Detect-resolve-forget is the right pattern for design goals where collapsing the alternative space before emission is the intent and where the discarded alternatives carry no value the system is responsible to preserve. The CKS commitment is the right pattern for coordination settings where the dispute record is itself part of what the substrate exists to carry. The two stances take opposite positions on what the substrate is for.

4. Provenance requirements for first-class contradictions

For a contradiction to count as a first-class object in the CKS sense, the substrate must carry, for each piece of contradicting content, four properties:

1. **The writer.** The actor that produced the content — human, or LLM operating under a specific orchestration rule. Where the writer is an LLM, the rule the LLM was operating under is part of the writer record.
2. **The timestamp.** When the content was written into the substrate.
3. **The rationale,** where applicable. The reason the writer offered for the content, including any references to other substrate content the rationale rests on.
4. **The relationship to the contradicting content.** Which other piece or pieces of substrate content the content contradicts, and on what dimension the contradiction holds.

The first three properties are provenance properties general to substrate content; the human-governed commitment and the path-retraceability vocabulary already require them for any auditable substrate. The fourth property — explicit relationship — is what distinguishes a contradiction-as-first-class-object from a mere accumulation of inconsistent content. Two pieces of substrate content can be inconsistent with each other without that inconsistency being addressable; addressability requires that the relationship between them be substrate content in its own right, not a property a reader has to reconstruct by reading both.

The strongest 2024–2026 adjacent precedent for treating the contradiction *relationship* as addressable substrate state is OIDA's *signed contradiction edges*, cited in §5.2 and §8.2 of the source paper. A signed contradiction edge between two Knowledge Objects has its own schema presence, points to both conflicting objects, and persists in the substrate rather than existing as transient state consumed in a single inference pass. The architectural move OIDA makes — promoting the contradiction relationship to first-class substrate state, on the same terms as the contradicting objects themselves — is the precedent the fourth provenance requirement above draws on. The source paper is explicit that OIDA validates the design pattern rather than scoops it: CKS differs from OIDA on the human-governance, role/authority, and AI-as-substrate-mediator axes (§5.2), but on the narrower question of whether contradiction relationships should be substrate-addressable, OIDA and CKS commit to the same

answer. Implementations that omit the fourth requirement and treat contradictions as merely co-resident substrate content miss the architectural difference between holding inconsistency and recording it as a navigable structure.

5. Operational test

A system implements the CKS conflict-as-first-class-object commitment if and only if all of the following are true at all times during the substrate's existence:

1. Contradictions in substrate content are recorded as substrate content, not as errors, warnings, or transient signals consumed during inference.
2. Each piece of contradicting content carries writer, timestamp, rationale (where applicable), and explicit relationship to the content it contradicts (which other content, on what dimension).
3. No LLM operation, automated process, or runtime middleware layer can silently collapse a contradiction. Collapse requires either direct human action or a human-authored orchestration rule explicitly authorizing it for the specific case.
4. Cell-level resolution decisions are themselves recorded as substrate content, with their own writer, timestamp, and reference to the orchestration rule that authorized the resolution.
5. Substrate-level contradictions persist after cell-level resolution decisions; the resolution does not mutate, overwrite, or remove the underlying contradicting content.

A system that fails any of (1)–(5) may handle contradictions in some other useful way, but does not instantiate the CKS conflict-as-first-class-object commitment. A system that satisfies (1) and (2) but not (3) is a logging system, not a preservation system; a system that satisfies (3) but not (4) loses the audit trail that makes preservation meaningful; a system that satisfies (4) but not (5) records its own collapse decisions while still collapsing.

6. Why naming both halves matters for downstream work

Two failure modes recur in implementations of architectures adjacent to CKS, and naming both halves of the commitment together is what guards against them.

The first failure mode is conflating conflict preservation with conflict logging. An implementation in this mode records contradictions as historical events in an append-only log, capturing that a contradiction occurred and who its participants were, but not carrying the contradiction as live substrate state. Cells executing over the substrate read only the resolved or surviving content; the log exists for retrospective audit only. This satisfies a weak reading of "preserved" but not the architectural commitment, because the contradiction is not addressable substrate content — it is history *about* substrate content, recorded outside the substrate's working state. The substrate must carry the contradiction *as state*, not as history.

The second failure mode is conflating two-level handling with single-level escalation: every contradiction blocks until a human resolves it. Implementations in this mode preserve contradictions at the substrate level but provide no rule-governed cell-level pathway for action in their presence; cells

either halt or surface the contradiction to a human reviewer for every encounter. This honors substrate-level preservation but breaks the operational tractability the second level provides. Cells must be able to act in the presence of active contradictions, under rules humans authored at design time, without forcing collapse and without forcing escalation on every occurrence.

The two failure modes are symmetric: the first half of the commitment without the second produces a substrate that cannot be acted on; the second without the first produces cells that act but leave no record. Naming both halves together — substrate-level preservation as live state, cell-level resolution under orchestration rules, with the resolution itself recorded as substrate content rather than as mutation — is what makes the pattern stable. Subsequent work that adopts, extends, composes, or argues against the commitment should use "conflict-preserving" in the sense formalized here. Subsequent work that uses the term differently is using a different commitment, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Conflict as First-Class Object: Two-Level Conflict Handling in the Coordination Knowledge Substrate Pattern*. 24 April 2026. ORCID: 0009-0004-8065-3235.